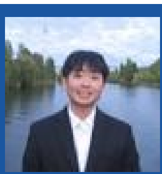

Python File I/O

Data with Dylan | Guided Notes | Python Fundamentals #6

Name: ______________________________

Date: ______________________________

☑ HOW TO USE THESE NOTES: Watch the video and fill in the blanks as Dylan explains each concept. Use the writing boxes to capture your own notes and examples. Complete the practice problem at the end!

Section 1: File I/O Basics

Topic: Saving data so it can be used again later

What is File I/O?

Complete the definition:

File I/O stands for file ________________ and ________________.

File I/O lets a Python program save data _____________________ or access data __________________ a file.

A file stream is a connection between a Python program and a _________________________.

The Three Main Steps

1. ____________________ a file
2. ____________________, write, or append data
3. ____________________ the file

Complete the basic example:

```
file = open("file.csv", "_______")

file.__________________("hello")

file.__________________()
```

In your own words - why is File I/O useful when working with data?

Section 2: Opening Files and File Modes

Topic: Choosing how Python should interact with a file

The open() Function

Complete the definition:

The open() function tells Python which ________________ to open and which _________________ to use.

A file mode tells Python whether you want to ________________, write, or append data.

File Mode Reference

MODE	MODE NAME	BEHAVIOR IF FILE EXISTS / MISSING
`r`	_______________	Reads an existing file. If it is missing, Python raises a _______________.
`w`	_______________	Replaces existing file contents. Creates a new file if one does not exist.
`a`	_______________	Adds new data to the end of an existing file. Creates a new file if one does not exist.

Compare Write and Append Modes

When a file is opened with "w", old data is _____________________.

When a file is opened with "a", old data is _________________ and new data is added to the _____________.

In your own words - what is the most important difference between "w" mode and "a" mode?

Circle the VALID context manager syntax

Choice A

```
with open("college_data.csv", "w") f:
    f.write("hello")
```

Choice B

```
with open("college_data.csv", "w") as f:
    f.write("hello")
```

Why is the other choice incorrect?

Section 3: Reading Text Files

Topic: Accessing saved information with read mode

Reading a File

Complete the definition:

Read mode uses the parameter "________".

The .read() method returns the ______________________ stored in a file.

A FileNotFoundError happens when Python tries to open a file that _________________________.

Complete the code below:

```
with open("notes.txt", "_______") as f:
    contents = f.__________________()

print(____________________)
```

Reading Safely with with

The with statement creates a _______________________ manager.

A context manager automatically handles opening and ______________________ a file.

In your own words - why is using with safer than manually calling .close()?

Predict the Result

CODE	WHAT HAPPENS?
`with open("notes.txt", "r") as f:`	____
`f.read()`	____
`open("missing_file.txt", "r")`	____

Section 4: Writing and Appending Data

Topic: Saving new data into files

Writing Data

Complete the definition:

The .write() method sends ______________________ into an open file.

Opening a file in "w" mode allows Python to _______________________ existing contents.

Complete the code:

```
with open("message.txt", "w") as f:
    f.__________________("Hello!")
```

After this code runs, the file should contain: ____

Appending Data

Complete the code:

```
with open("message.txt", "a") as f:
    f.__________________("\nAnother message")
```

The \n creates a new ________________.

Appending is useful when you want to keep existing information and add a ______________________ record.

In your own words - when would you choose append mode instead of write mode?

File Operation Reference

CODE	METHOD / OPERATION NAME	RESULT / NOTES
`open("data.txt", "r")`	________________	Opens a file for reading.
`open("data.txt", "w")`	________________	Opens a file for writing and replaces old contents.
`open("data.txt", "a")`	________________	Opens a file and adds new content at the end.
`f.write("hello")`	________________	Writes text into the file.
`f.close()`	________________	Closes the file connection.

Section 5: Writing Dictionary Data to a CSV File

Topic: Converting organized dictionary data into a CSV format

Preparing the Data

A CSV file stands for ____________________________ separated values.

The csv module provides tools for working with ___________________ files.

Complete the import:

```
import _______
```

The dictionary keys represent the CSV file's ____________________.

The lists stored as dictionary values contain individual ____________________ points.

```
college_data = {
    "university": ["UW", "WSU"],
    "in_state_tuition": [13406, 12601],
    "acceptance_rate": [42.5, 76.0]
}
```

Creating a Context Manager for CSV Output

Complete the syntax:

```
with open("college_data.csv", "_______", newline="") as _______:
```

The newline="" parameter helps prevent extra __________________ between CSV rows.

Setting Up the Writer

```
fields = ["university", "in_state_tuition", "acceptance_rate"]

writer = csv._________________(f, fieldnames=fields)
```

The fieldnames parameter identifies the CSV _______________________.

Complete the header-writing method:

```
writer.__________________()
```

In your own words - why should a CSV file include column headers?

Section 6: Building and Writing Each CSV Row

Topic: Matching dictionary values to their correct headers

Finding the Number of Rows

Complete the code:

```
num_rows = len(next(iter(college_data.__________________())))
```

This line gets the length of one list because each list contains the same number of __________________.

Constructing Each Row

```
for i in range(num_rows):
    row_dictionary = {}

    for key in college_data.__________________():
        row_dictionary[key] = college_data[key][_______]

    writer.__________________(row_dictionary)
```

The outer loop moves through each _________________ number.

The inner loop moves through each dictionary _______________.

The row_dictionary stores data for one complete __________________.

The .writerow() method maps each value to the matching CSV _________________.

Trace One Row

DICTIONARY KEY	VALUE AT INDEX 0	VALUE ADDED TO THE ROW
`"university"`	________________	________________
`"in_state_tuition"`	________________	________________
`"acceptance_rate"`	________________	________________

In your own words - why does the row dictionary reset to {} during every outer-loop iteration?

Section 7: Putting It All Together

Topic: File I/O reference table

Using what you have learned, complete the full file operations reference table:

FILE OPERATION	METHOD USED	MINIMAL CODE EXAMPLE	YOUR OWN EXAMPLE
Open a file for reading	open() with "r"	`open("notes.txt", "r")`	
Read all file contents	read()	`f.read()`	
Open a file for writing	open() with "w"	`open("data.txt", "w")`	
Replace old file contents	write()	`f.write("hello")`	
Add information to a file	open() with "a"	`open("data.txt", "a")`	
Close a file manually	close()	`f.close()`	
Open and close automatically	with statement	`with open(...) as f:`	
Write CSV headers / one row	writeheader() / writerow()	`writer.writeheader()` `writer.writerow(row)`	

Section 8: Quick Recap

Test yourself - try to complete these without looking back!

1. File I/O stands for file ______________________________ and ______________________________.

2. The three main steps are open, ______________________________ or change data, and ______________________________.

3. Opening a file with "w" mode will ______________________________ existing contents.

4. Opening a file with "a" mode adds new data to the ______________________________ of a file.

5. A with statement is useful because it automatically ______________________________ the file when the code block ends.

Practice Problem

⚡ **Challenge - Pause the video and try this yourself before Dylan reveals the answer!**

```
import csv
new_record = {"university": "UW", "in_state_tuition": 13406, "acceptance_rate": 42.5}
```

Goal: Create a script that safely opens a local CSV file, writes or appends new_record, and reads it back out to the terminal.

- Use a with open(...) as f: context manager and csv.DictWriter.
- Use "w" to create a fresh file or "a" to preserve earlier records.

Why? Why is a context manager safer than manually opening and closing your file?

Why? Why might append mode be a better choice than write mode when the file already contains previous records?

Write your full statement below:

How did it go? What part tripped you up?

Personal Notes

Use this space to capture any additional observations, questions, system path questions, or ideas from the video.

Key Terms Reference

Use this table to build your Python vocabulary. Write definitions in your own words.

TERM	DEFINITION IN YOUR OWN WORDS
File I/O	
File	
File Stream	
File Path	
open()	
File Mode	
Read Mode (r)	
Write Mode (w)	
Append Mode (a)	
read()	
write()	
close()	
with Statement	
Context Manager	

Key Terms Reference (continued)

Continue building your Python vocabulary. Write definitions in your own words.

TERM	DEFINITION IN YOUR OWN WORDS
as Keyword	
FileNotFoundError	
newline=""	
CSV	
Comma-Separated Values	
csv Module	
csv.DictWriter	
fieldnames	
writeheader()	
writerow()	
Dictionary	
Row Dictionary	
Column Header	
for Loop	
Indexing	